ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/360864737

K2E: Building MLOps Environments for Governing Data and Models Catalogues while Tracking Versions

Conference Paper · March 2022

DOI: 10.1109/ICSA-C54293.2022.00047

CITATIONS
9

READS
52

4 authors:

Gorka Zarate
Tecnalia
8 PUBLICATIONS **84** CITATIONS

SEE PROFILE

Raúl Miñón
Tecnalia
35 PUBLICATIONS **524** CITATIONS

SEE PROFILE

Josu Diaz de Arcaya
Tecnalia
24 PUBLICATIONS **235** CITATIONS

SEE PROFILE

Ana-Isabel Torre-Bastida
Tecnalia
40 PUBLICATIONS **500** CITATIONS

SEE PROFILE

All content following this page was uploaded by Gorka Zarate on 13 February 2024.

The user has requested enhancement of the downloaded file.

K2E: Building MLOps Environments for Governing Data and Models Catalogues while Tracking Versions

Gorka Zárate*, Raúl Miñón†, Josu Díaz-de-Arcaya‡ and Ana I. Torre-Bastida§
TECNALIA, Basque Research and Technology Alliance (BRTA), Vitoria-Gasteiz,Spain
Email: *gorka.zarate@tecnalia.com, †raul.minon@tecnalia.com, ‡josu.diazdearcaya@tecnalia.com, §isabel.torre@tecnalia.com

***Abstract*—Nowadays, the data management processes and flows utilised for extracting value and information from the data must deal with various problems. In particular, the data heterogeneity, the distribution of the data, and the vast variety of techniques and approaches make data management hard to implement. In addition, the management of data and analytical models is still a challenge to be solved. In this context, the catalogue tools for data and Artificial Intelligence models alleviate the burden of dealing with data and model versioning. Thus, the automation of the data and models' management processes is facilitated boosting paradigms like DataOps and MLOps. This work in progress enumerates key challenges to address when creating these types of catalogues: on the one hand, the management of the diversity of data and models' internal nature and their different versions, and on the other hand, the provision of adequate meta-information and Governance tools such as access control and auditory. In this area, the Knowledge to Environment (K2E) platform is presented, whose architecture aims to define the necessary components for the creation of environments that allow working with data and model catalogues. By environment creation, we understand providing a workspace populated with the datasets and models of an organization, while tracking their distinct versions by using specialised catalogues. In addition, this workspace will incorporate added-value tools for governance and auditing. Finally, an approach for implementing K2E is detailed.**

***Index Terms*—data, dataset, datalake, models, catalogues, metadata, DataOps, MlOps, management, versioning, automation**

I. Introduction

Currently, the challenges associated with AI operationalisation, that is, deploying AI processes in real production environments are numerous. The operations are similar to any traditional IT project: build / test / deploy / manage, but with two operational perspectives: training and inference and two component perspectives: data and models. In this context, the MLOps paradigm is envisioned as a promising solution.

MLOps can be understood as a set of processes or rather a sequence of steps implemented to deploy a ML model in a production environment. These models must be previously trained (development) based on a dataset and then, the resulting model can be used to make inferences (operation). And in this context, how do we manage to adequately govern this experimentation and the data that is used? In the arsenal of MLOps capabilities, an experimentation catalogue has moved from a "nice to have" to a "must have" due to the growing importance of AI processes as data driven business enabling technologies.

In our opinion, the Machine Learning experimentation governance must be constructed based on two key tasks: data catalogue and model catalogue. In this paper, a data catalogue is defined as a detailed and organized inventory of all data assets in an organization, and the model catalogue can be considered as an experiment tracking system for machine learning. The union of both technologies allows the correct management and reproducibility of the AI processes deployed in organizations, avoiding the loss of knowledge and information. Nowadays, it is an utopia to implement this holistic vision of the AI experiments governance using a single tool or platform. From both the data perspective and the replication of the experiments, there are multiple dedicated tools, but none encompasses both tasks completely.

In this paper, our contribution analyses this gap of tools for MLOps and especially for the task of cataloguing the whole experiment life-cycle. Moreover, a platform architecture capable of implementing this MLOps experiment catalogue completely, along with the necessary dataset versioning associated is proposed. Another feature to highlight of this platform is the functionality of integrating existing datasets or AI processes without the need to perform a complex reengineering process. The preliminary approach in the implementation of the architecture, which has been based on the DVC [3] tool, is also detailed.

The paper is structured as follows: Section II portrays current challenges in the field of experiment tracking and data versioning research. Subsequently, the related work is discussed in Section III. In Section IV, the architecture and the approach for the implementation is presented. Finally, in Section V the conclusions and future work are presented.

II. Challenges

The conceptualization of data and model catalogues to boost the MLOps process entails a set of challenges that must be carefully addressed. In this section, the main considerations to take into account are examined:

- **Diversity management:** there exist an extensive number of techniques and formats for storing datasets ranging from specific files such as CSVs, JSONs or a folder with images to more sophisticated systems such as traditional

or distributed databases. In addition, data can be allocated near the data catalogue (in the same cluster, following for instance a datalake architecture) or remotely in a set of different servers. When dealing with model catalogues the situation is not that different since to characterise adequately a specific model, aspects concerning the model training algorithms and frameworks, evaluation metrics and serialization formats are crucial.

- **Version control management:** datasources and models could be used in different contexts or projects which might originate different versions of them. For instance, new images included in a datasource could not be applicable for a specific project. Identically, when creating a model suitable for a specific domain, it may require some tuning or modifications to be useful for a different one. However, despite having some changes over datasets or models, in essence, they constitute the same element. Therefore, it is highly recommended to integrate a mechanism able to deal with distinct versions of datasets and models enabling to relate different versions as the same element, as well as facilitating the change among the available versions. In this way, data scientists and data engineers would be able to easily access a desired version, achieving the reproducibility of the desired experiment. At this point, it is worth mentioning that dealing with large files is not trivial when proposing a version control management system.
- **Metainformation definition:** datasets and models registered in a catalogue must be provided with key metainformation in order to better characterise them and to provide additional information. This way, a functionality for discovering datasets and models would be drastically enhanced. Moreover, MLOps automated processes can benefit from this metainformation to adequately fulfil their objectives. As a matter of example, besides considering the dataset type, model algorithm and framework and other contextual information, it could be useful information such as the location, the size, creation date, whether the data or the model expires and when, the owner, projects involved, domains or associated regulation policies.
- **Data and model governance:** these catalogues should provide added value when governance is concerned. First of all, they must be protected against inadequate accesses by providing strict authentication and authorization policies. In addition, it should be desirable the provision of audit and tracking tools (Data Lineage) in order to being able to easily identify who and when the data and the models have been utilised. Data Lineage describes the origin, movements, characteristics, and quality of the data. Moreover, as suggested before, metainformation is crucial to improve the discovery, but its performance could still be more precise by supporting an additional tags mechanism to associate extra fields than the fixed in the metainformation. Finally, more sophisticated tools could be considered for instance to periodically evaluate the correctness of the links or detect duplicated data and models (data quality) or to enforce data transformations such as anonymisations when specific regulations must be fulfilled (data usage).

III. Related Works

In this section, the state of the art with respect to different tools that aid data engineers and data scientists in the process of versioning some stages of the machine learning life-cycle is analysed. In particular, the versioning of the models and the data associated with them.

In Table I, a comparison of different tools that tackle the aforementioned problem is offered. These tools have been evaluated with focus on the following criteria: (i) it is distributed under an Open Source license, (ii) it allows the versioning of existing datasets in a native way, (iii) it supports the most common storage technologies both on premises and in the cloud, (iv) it allows the versioning of the experiment, (v) it comprises a set of tools to govern the experiment, (vi) it provides means to track the data lineage, (vii) it provides a search tool for experiments or datasets, (viii) it allows the creation of reproducible pipelines, (ix) it saves metainformation about datasets or models.

DVC [3] was created to maintain the models shareable and reproducible. However, it is not easy to use old datasets as it requires the repositories to be tracked beforehand. Next, Quilt [14] embraces the use of data as code, so teams can experiment and recover from errors faster, but to the best of our knowledge only supports a limited set of storage technologies. Valohai [17] aspires to let data scientists be focused more on the model's operationalization. However, it does not address various of the defined criteria such as not being Open Source or not allowing the versioning of existing datasets. Next, DAGsHUB [1] is able to integrate with existing tools such as DVC and MLFlow [12], but cannot work with existing datasets. Intake [9] is a library that can retrieve existing datasets, but does not support versioning, data governance, amongst others. A popular framework for experimenting with machine learning is Neptune [11], which provides a user interface for working with the experiments (e.g., training, versioning), but it is a closed solutions and it does not support the versioning of existing datasets. On the other hand, Verta [18] excels at the machine learning deployment in production environment, but lacks the ability to version such models, nor to manage the governance and lineage of the data. Guild [16] is an Open Source tool that lets you better understand the model and its associated metrics, although it does not provide means to perform data governance nor lineage. In summary, none of the analysed tools fulfils all the criteria showcased in Table I. Most of them are focused on covering the versioning part of the model catalogue, ignoring the versioning of the associated datasets. This generally makes reproducibility be low and very dependent on the good practices of data scientists. The main reason is that versioning binary files (e.g., images) is a cumbersome problem, because small changes in an image can lead to a big change in the

	(i) Open Source	(ii) Dataset Support	(iii) Storage Support	(iv) Version Control	(v) Data Governance	(vi) Data Lineage	(vii) Searchable Datasets	(viii) ML Pipeline	(ix) Metainformation
DVC [3]	✓	✗	✓	✓	✗	✗	✗	✓	✓
Quilt [14]	✗	✗	≈	✓	✓	✗	✓	✗	✓
Valohai [17]	✗	✗	✓	≈	✗	✓	✗	✓	✓
DAGsHUB [1]	✗	✗	✓	✓	✓	✗	✗	✓	✓
intake [9]	✓	✗	✓	✗	✗	✗	✓	✗	✗
Neptune [11]	✗	✗	✓	✓	✗	✗	✓	✗	✓
Verta [18]	✗	✗	✓	≈	✗	✗	✓	✗	✓
Guild AI [16]	✓	✓	✓	✗	✗	✗	✓	✓	✓
K2E	✓	✓	✓	✓	✓	✓	✓	✓	✓

✓: full support, ≈: partial support, i: not enough information, ✗: not supported

TABLE I

TOOLS THAT SUPPORT THE VERSIONING OF THE MACHINE LEARNING LIFE-CYCLE.

binary file itself. Due to this, a common approach for the versioning of binary files is the use of unique identifiers (e.g., hash functions). In order to fill this gap, we aspire to release a new tool, K2E, which is explained in further detail in Section IV.

IV. ARCHITECTURE

A. Overview

This section presents the conceptualization of an architecture, called Knowledge to Environment (K2E), aimed at designing the main building blocks required to support the creation of data and model catalogues. This approach is focused in addressing the following objectives derived from challenges posed in section II: Creation of centralised data and model catalogues with metainformation associated; Whole management of the distinct versions of datasets and models that can potentially be generated for different contexts, projects or experiments; Provision of a mechanism for the governance and auditing of the datasets and models; and Open conceptualization to enable the future integration of new connectors for dealing with the diversity.

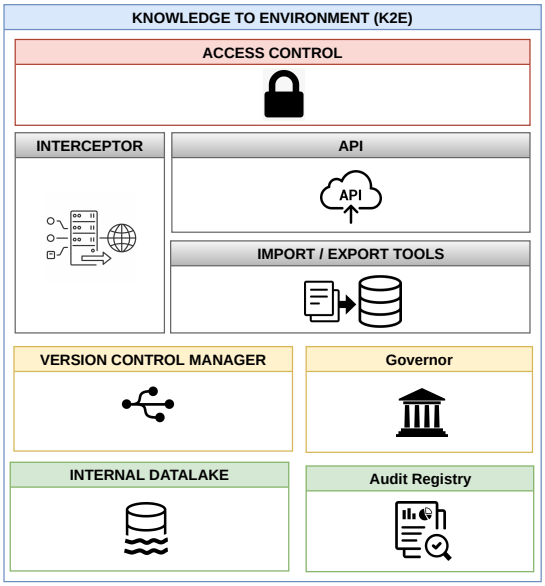

Fig. 1. Abstract architecture where the main building blocks are presented

Figure 1 exposes the identified building blocks for creating data and model catalogues addressing these objectives. Firstly, there is an access layer conformed by i) the **Access Control** which is in charge of managing the authentication and authorisation policies allowing access to registered users with their associated permissions; ii) the **API** for allowing the interaction with the platform providing a set of endpoints ranging from security tasks to data and model related operations; iii) **Import and Export tools** is designed to enable the migration of external datasets and models to the platform, as well as facilitating the transfer of these entities to another systems; iv) the **Interceptor** allows to capture the incoming traffic to the Access control, API and Import/Export tools components to register every interaction with the platform.

Secondly, there is a processing layer where it is located i) the **Version Control Manager** dedicated to store datasets and models metainformation including their different versions and pointers to the specific location where the datasets and models are really located in the Internal Datalake. This approach allows to keep track of the different dataset and model variations among their different versions as well as avoiding to store each file repeatedly and being able to manage large files. In addition, in this layer, the **Governor** is dedicated to deal with specific data and model governance aspects. Currently, besides the access control support, the platform has been conceptualized to support auditing mechanisms. To this end, this module is fed by the data captured by the Interceptor in order to prepare it for being stored in the Audit Registry. If a suspicious operation were conducted, an email is sent to the system administrator.

Finally, the storage layer includes two components already mentioned, i) the **Internal Datalake** for storing the large files managed by the catalogues, in diverse formats, such as datasets or heavy models; and ii) the **Audit Registry** to save all the interactions done with the platform including fields such as the timestamp, the user, the operation performed and the normality of the operation (normal, occasional, or suspicious).

At this point, it is worth noting that, currently, there is no possibility to integrate in the catalogues external datasets without being replicated in the internal one. This is due to the fact that, as far as we are concern, there is no existing tool being able to track the versions of already existing datasets. However, this feature will be explored in depth

in the future work. On the other hand, instead of being a disadvantage, the datalake paradigm allows to collect all the data and models from a company in centralised fashion. This approach is typically followed by Big Data platforms bringing some benefits such as the fast processing of data, the ability to manage large volumes and heterogeneous data, the high availability, a performance improvement when merging data and a unified and standardised access to the data and models.

B. Platform Envisioning

This subsection presents key ideas concerning how to implement certain building blocks of K2E. Figure 2 exposes the technologies prescribed for this objective. For this purpose, different types of datasources and datasets stored on them are considered such as S3, Azure, Databases, Local, etc.

K2E users will interact with a Python 3 [13] Flask [4] API with OpenApi [15] support to perform certain tasks such as the registration of datasets and models, including metainformation, the management of distinct versions or the triggering of import and export processes. This last task involves the migration of these external existing datasets to K2E, which will be also developed with Python for having a vast set of libraries to work with data and models. Besides centralizing the information, DVC [3] will be used to build the version control manager system since it provides a mechanism to manage the versions using Git [5] as underlying system. In addition, GitLab [6] have been selected as Git repository since it is a mature system which provides a complete API to interact with. In this way, the ultimate goal of providing a working environment to start exploiting datasets and models in a managed way can be accomplished.

It is worth mentioning that the environments and the whole platform will be built by following the Infrastructure as Code paradigm. As a consequence, Docker [2] containers will encapsulate the distinct building blocks, Terraform [7] will be in charge of the provisioning of new environments and Ansible [8] will be responsible for the configuration of the required software libraries and dependencies.

Despite not being included in this iteration, multi-user teams will be able to bifurcate projects and compare results delegating the duties of authentication and authorisation to KeyCloak [10]. Thus, alongside with the environment internal functionalities, audit and governance mechanisms will be improved, capturing, auditing, and managing the different accesses and actions performed over the datasets and models.

V. Conclusions and Future Work

In this article we present K2E, a software platform for the versioning of models and data. Due to the exhaustive study of the state of the art conducted and the identification of the key challenges, we have been able to design a complete architecture taking the gaps observed in the fields of cataloguing and versioning of data and models into consideration. Furthermore, we propose an Infrastructure as Code driven approach for conducting key implementation aspects, which covers the data

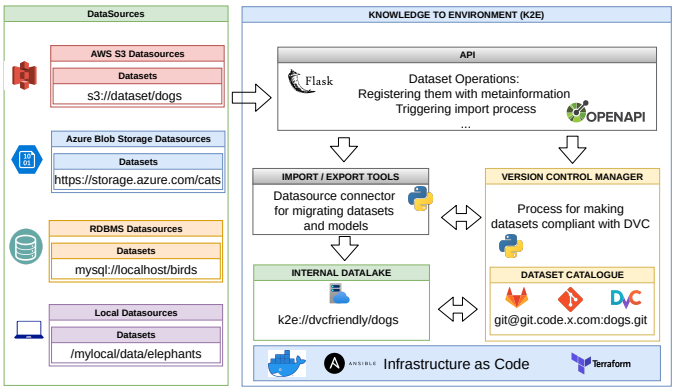

Fig. 2. Implementation of the conceptual architecture, K2E

catalogue capabilities improving an existing versioning tool such as DVC.

Regarding future work, besides implementing the whole platform, we would be interested in improving our data catalogue features with the aim of dealing with the versioning on previous existing datasets in organizations, without the need to change the data storage and management mechanism or philosophy. Additionally, the inclusion in K2E of Infrastructure as Code (IaC) functionalities to be able to provide the necessary storage infrastructural resources for the data and model versioning. In additions, the dynamic and automatic deployment and adaptation of these resources are also in our roadmap to cover appropriate service and performance levels.

Acknowledgment

The work presented in this paper has been partially supported by the SPRI Basque Government through their ELKARTEK program (DAEKIN project, ref.KK-2020/00035).

References

[1] DAGsHUB. Dagshub, the home for data science collaboration, 2021.
[2] Docker. Docker, 2021.
[3] DVC. Data version control, dvc., 2021.
[4] Flask. Flask, 2021.
[5] Git. Git, fast version control, 2021.
[6] GitLab. Gitlab: iterate faster, innovate together, 2021.
[7] HashiCorp. Terraform, 2021.
[8] Red Hat. Ansible, 2021.
[9] Intake. Intake, 2021.
[10] Keycloak. Keycloack: Open source identity and access management, 2021.
[11] Neptune Labs. Netptune.ai, metadata store for mlops, 2021.
[12] LLC LF Projects. Mlflow, 2021.
[13] Python. Python 3, 2021.
[14] Inc Quilt Data. Quilt, 2021.
[15] Swagger supported by Smartbear. Openapi specification, 2021.
[16] Inc TensorHub. Guild ai, experimet tracking, ml developer tools, 2021.
[17] Valohai. Valohai mlops platform, 2021.
[18] Verta. Verta, model management and operations for mlops and modelops, 2021.

View publication stats